



PDF Download
3570926.pdf
31 March 2026
Total Citations: 7
Total Downloads: 703

Latest updates: <https://dl.acm.org/doi/10.1145/3570926>

RESEARCH-ARTICLE

GANDSE: Generative Adversarial Network-based Design Space Exploration for Neural Network Accelerator Design

LANG FENG, Nanjing University, Nanjing, Jiangsu, China

WENJIAN LIU, Nanjing University, Nanjing, Jiangsu, China

CHULIANG GUO, Zhejiang University, Hangzhou, Zhejiang, China

KE TANG, Nanjing University, Nanjing, Jiangsu, China

CHENG ZHUO, Zhejiang University, Hangzhou, Zhejiang, China

ZHONGFENG WANG, Nanjing University, Nanjing, Jiangsu, China

Open Access Support provided by:

Nanjing University

Zhejiang University

Published: 19 March 2023
Online AM: 09 November 2022
Accepted: 31 October 2022
Revised: 05 October 2022
Received: 23 July 2022

[Citation in BibTeX format](#)

GANDSE: Generative Adversarial Network-based Design Space Exploration for Neural Network Accelerator Design

LANG FENG and WENJIAN LIU, Nanjing University, China
CHULIANG GUO, Zhejiang University, China
KE TANG, Nanjing University, China
CHENG ZHUO, Zhejiang University, China
ZHONGFENG WANG, Nanjing University, China

With the popularity of deep learning, the hardware implementation platform of deep learning has received increasing interest. Unlike the general purpose devices, e.g., CPU or GPU, where the deep learning algorithms are executed at the software level, neural network hardware accelerators directly execute the algorithms to achieve higher energy efficiency and performance improvements. However, as the deep learning algorithms evolve frequently, the engineering effort and cost of designing the hardware accelerators are greatly increased. To improve the design quality while saving the cost, design automation for neural network accelerators was proposed, where design space exploration algorithms are used to automatically search the optimized accelerator design within a design space. Nevertheless, the increasing complexity of the neural network accelerators brings the increasing dimensions to the design space. As a result, the previous design space exploration algorithms are no longer effective enough to find an optimized design. In this work, we propose a neural network accelerator design automation framework named GANDSE, where we rethink the problem of design space exploration, and propose a novel approach based on the generative adversarial network (GAN) to support an optimized exploration for high-dimension large design space. The experiments show that GANDSE is able to find the more optimized designs in negligible time compared with approaches including multilayer perceptron and deep reinforcement learning.

CCS Concepts: • **Hardware** → **Electronic design automation**; • **Computing methodologies** → **Machine learning**;

Additional Key Words and Phrases: Design space exploration, generative adversarial networks

ACM Reference format:

Lang Feng, Wenjian Liu, Chuliang Guo, Ke Tang, Cheng Zhuo, and Zhongfeng Wang. 2023. GANDSE: Generative Adversarial Network-based Design Space Exploration for Neural Network Accelerator Design. *ACM Trans. Des. Autom. Electron. Syst.* 28, 3, Article 35 (March 2023), 20 pages.
<https://doi.org/10.1145/3570926>

This work was partially supported by National Key R&D Program of China (Grant No. 2018YFE0126300), National Natural Science Foundation of China (Grants No. 62204111, No. 62034007, and No. 62141404), Zhejiang Provincial Key R&D program (Grant No. 2020C01052), and Shuangchuang Program of Jiangsu Province (Grant No. JSSCBS20210003).

Authors' addresses: L. Feng, W. Liu, K. Tang, and Z. Wang, Nanjing University, 163 Xianlin Road, Nanjing, China, 210023; emails: {flang, wjliu}@nju.edu.cn, mf21230105@smail.nju.edu.cn, zfwang@nju.edu.cn; C. Guo, Zhejiang University, 866 Yuhangtang Road, Hangzhou, China, 310058; email: chuliang007@zju.edu.cn; C. Zhuo (corresponding author), Zhejiang University, 866 Yuhangtang Road, Hangzhou, China, 310058 and Key Laboratory of Collaborative Sensing and Autonomous Unmanned Systems of Zhejiang Province, Hangzhou, China, 310027; email: czhuo@zju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Association for Computing Machinery.

1084-4309/2023/03-ART35 \$15.00

<https://doi.org/10.1145/3570926>

1 INTRODUCTION

Deep learning is prevalent in the area of **artificial intelligence (AI)** due to its success in many fields such as vision and speech processing. Initially, the training and inference of deep learning were deployed on the **central processing unit (CPU)**. After exploring the parallel computing capability of the **graphics processing unit (GPU)**, GPU became the most common platform for deep learning [4]. However, with the growth of the deep learning model size, using GPU can incur large power consumption that is not acceptable for many low power applications, such as edge computing.

To control the power consumption while maintaining the performance of executing large neural network models, the dedicated hardware neural network accelerators are proposed, such as Eyeriss [3], Gemmini [6], DaDianNao [11], and so on. However, deep learning algorithms keep updating frequently. As designing the hardware accelerators requires a long period, frequent algorithm updating can introduce significant engineering effort and cost. To avoid this, researchers try to seek the design automation approaches for neural network accelerators.

There are various studies of neural network accelerator design automation, such as DnnWeaver [17], DNNBuilder [22], AutoDNNchip [20], ConfuciusX [9], and so on. Each of them proposed an automatic design framework to search and generate the optimized accelerator, given a deep learning inference task. Given the requirements from the user, the searching problem to get the optimized design that satisfies the requirements is called the **design space exploration (DSE)** problem, which is a step in neural network accelerator design automation. However, each of the previous DSE algorithm [1, 9, 10, 17, 18, 20, 22, 23] iteratively uses a model to evaluate the quality of the searching result, and use a searching algorithm to update the searching result given the evaluation. The iterative searching can incur large searching time cost. Recent study [15] proposes a non-iterative approach by using **multilayer perceptron (MLP)** for DSE, but our experiments show that MLP is still not competent under high-dimension design space. Therefore, a new algorithm that is able to effectively search the optimized design in a high-dimension large design space is needed.

The neural network accelerator design automation is to generate a neural network accelerator, given the objectives from the designers. This is similar to a kind of neural networks called **generative adversarial networks (GANs)**, which are usually used to generate images that satisfy the requirements from the users. Inspired by this, we investigate the potential to apply GAN to DSE tasks, and propose a GAN-based DSE flow. Because GAN usually has millions of outputs for image pixels, when using GAN for outputting the configurations, it has the potential to support high-dimension large design space. Besides, the iterative searching is also avoided.

In our work, we propose GANDSE, which is a neural network accelerator design automation framework with the GAN-based DSE. Different from previous work, GANDSE introduces a new DSE flow that can effectively find more optimized design within a high-dimension large design space. To the best of our knowledge, this is the first work that explores the potential to applying GAN in the field of DSE. The detailed contributions of this work are in the following.

- A neural network accelerator design automation framework named GANDSE is proposed to automatically generate the synthesizable RTL code of the optimized neural network accelerator design given a deep learning task from the user. The generated code can be implemented on an FPGA.
- A new DSE flow based on GAN is proposed, which provides a new direction for handling DSE problems, and the experiment results indicate the effectiveness.
- To fit our neural network accelerator design automation problem, the modifications are proposed to the structure of GAN, the training flow, the loss function design, the encoding of the features, and the candidate selection after the inference, and so on.

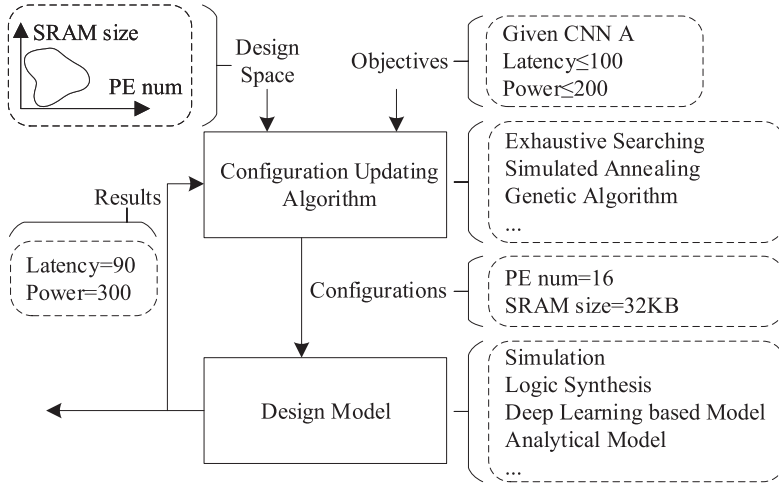


Fig. 1. Ordinary DSE flow and examples of neural network accelerator design automation.

- Throughout experiments are performed, which prove the effectiveness of the DSE algorithm in GANDSE. Compared with other approaches including simulated annealing, multilayer perceptron, and deep reinforcement learning, GANDSE finds more designs that satisfy the user's requirements in negligible time.

In the following sections, Section 2 introduces the background of design space exploration and generative adversarial networks. Section 3 discusses the related work. Section 4 introduces the motivation and design rationale of GANDSE. The overview of GANDSE is introduced in Section 5, and the design details are then proposed in Section 6. After that, the experimental results are analyzed in Section 7. Finally, Section 8 concludes the article.

2 BACKGROUND

2.1 Ordinary Neural Network Accelerator Design Space Exploration

DSE is the approach to search for the design, which satisfies the given **objectives**, in the design space. For example, the DSE discussed in this work is searching for a set of **configurations** of the neural network accelerators (such as the number of the **processing elements (PEs)**, the size of SRAM, etc.), which makes the accelerators satisfy some objectives such as the requirements to the latency, the power consumption, and so on.

According to the survey in Reference [16], most of the DSE algorithms can be illustrated in Figure 1. Two critical components are the **configuration updating algorithm** and the **design model**. The design model is a model of the metrics in the objectives. Given a set of configurations, the design model is able to output the values of the metrics (Note that for easy of discussion, in the following paper, the values of the metrics in the objectives such as “Latency = 90 cycles” are also referred as “objectives”). The design model can be constructed in various ways, including the analytical model, the deep neural networks, and so on. For the configuration updating algorithm, it is to output a set of the configurations, given the design space, the user's objectives, and the latest results of the metrics in the objectives. Some typical algorithms are exhaustive searching, simulated annealing, genetic algorithms, and so on.

However, the DSE flow in Figure 1 greatly depends on iteratively searching the optimized design, which is relatively less effective. In contrast, to avoid the iterative searching and explore the new direction for DSE, we propose a completely new DSE flow based on GAN in GANDSE.

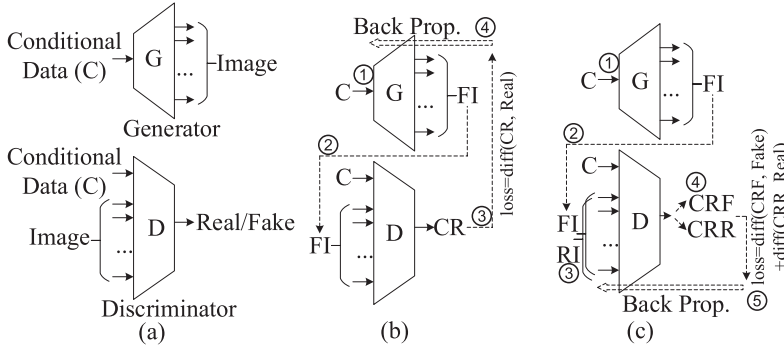


Fig. 2. (a) Example of a cGAN's architecture; (b) training procedures of G; (c) training procedures of D. (Note that G has an additional input with small random numbers as noise, which is not shown in this figure.)

2.2 Generative Adversarial Networks

GANs [7] are the neural networks that are able to generate new data, which have the same distribution as the training data. Specifically, **conditional GANs (cGANs)** [12] can generate the new data given some specifications, which are called the conditional information. For example, a cGAN can be trained to generate new images of specific kinds of animals with specific colors required by the users. Two critical neural networks involved in cGAN are called the **generator (G)** and the **discriminator (D)**. The generator can generate the new data while the discriminator can tell if the data is real (from the training set) or fake (generated from the generator).

Figure 2 shows an example of using cGAN for image generation. Given the conditional data, G tries to generate an image satisfying the condition and confuse D, while the goal of D is to discriminate if an image is a real image or a fake image. When training G, G is given the conditional data (C) of the images in the training set, and outputs the generated **fake images (FI)** accordingly. The fake images are then input to D, and the output **classification results (CR)** indicate if the images are real or not. The loss of G is to measure the difference between CR and “Real,” since it means G can generate the images that seem real enough if D classifies FI as “Real.” This loss is used to do the back propagation of G. When training D, the **real images (RI)** from the training set and FI generated from G are input to D. Then, D will output the classification results CRR and CRF on the real images and the fake images, respectively. The loss of D is to measure the difference between CRF and “Fake,” and that between CRR and “Real,” which represent the precision of the classification. Then, this loss is used for the back propagation of D. As the relationship between the image and the conditional data is similar to that between the accelerator configurations and the user's objectives, cGAN might also be used for the DSE. Because cGAN usually has millions of outputs for pixels, when using cGAN for outputting the configurations, it has the potential to support high-dimension large design space. Besides, since only one-time inference for each configuration is needed, it may save the time cost compared with running the iterative DSE algorithms.

3 RELATED WORK

With the development of neural networks, many neural network accelerators are also proposed. For example, Eyeriss [3] was designed for the energy efficient inference of **convolutional neural networks (CNNs)**, with its proposed dataflow. NVDLA [13] is a more general neural network accelerator that is configurable and supports IoT devices. Gemini [6] can generate the systolic arrays for matrix multiplication, and it can be integrated into RISC-V processors. DaDianNao [11]

is a multi-chip machine-learning architecture for realizing high-degree parallelism of neural network inference and training. Besides, Cambricon-Q [24] was also proposed for neural network training. It supports the efficient quantized training, and also has negligible accuracy loss. These proposed accelerators are manually well-designed, but the algorithms for neural networks evolve quickly, and manually designing the hardware accelerators can introduce significant engineering effort and cost. Different from these studies, the purpose of GANDSE is to automatically generate the accelerators that perform well enough to reduce the design cost.

There have been various studies about neural network accelerator design automation. Two typical design frameworks are DnnWeaver [17] and DNNBuilder [22]. DnnWeaver proposes an end-to-end design framework for generating the RTL code of the neural network accelerator, given the **deep neural network (DNN)** model written by the users using a proposed domain-specific language. The neural network accelerator is generated by setting the parameters of a Verilog template. DNNBuilder is similar to DnnWeaver, except that the template of DNNBuilder contains multiple systolic arrays, each of which processes one layer of the calculation of DNN inference. Meanwhile, AutoDNNchip [20] provides multiple detailed IP templates and global architectures that are composed of the chosen IPs. It expands the design space with higher granularity. DNNExplorer [23] uses a divide and conquer method for the DSE. In Reference [21], a fast DSE algorithm is proposed for searching the optimized size of the FFT for CNN calculations. The optimized size is found by calculating the intersections of the curves of the communication bound and computation bound. To avoid the iterative searching, AIRCHITECT [15] uses MLP for generating the accelerator design. However, our experiments show that only applying MLP finds limited optimized results given high-dimension large design space. ConfuciuX [9] uses reinforcement learning and the genetic algorithm for searching the optimized dataflow and hardware resource assignment. NAAS [10] proposes a design framework integrating the searches for the neural network architectures, the mapping methods, and the neural network accelerators.

Besides the approaches using the templates, using **high-level synthesis (HLS)** tools can also generate the neural network accelerators. AutoDSE [19] proposes a DSE approach that uses the bottleneck-guided gradient optimizer to find the bottleneck of the design and optimize it accordingly. The work [1] optimizes the HLS configurations with the use of exhaustive searching as the DSE algorithm. The work [18] uses the **graph neural network (GNN)** to increase the precision of the design model, and thus helps the quality of the DSE.

However, the DSE algorithms used in the previous studies related to neural network accelerator design automation either follow the ordinary DSE flow in Figure 1 or have low dimension design space, so they cannot effectively find the accelerator design that is optimized enough with the growing of the design space dimensions.

4 MOTIVATION AND DESIGN RATIONALE

In this section, we introduce the motivation and the core design rationale of GANDSE. The goal of the DSE for neural network accelerator design automation is that, given the user's objectives and the neural networks to be executed, a set of the configurations of the neural network accelerator needs to be generated, so that the configured neural network accelerator can satisfy the objectives.

Basically, to avoid the iterative searching of the ordinary DSE algorithms in Figure 1, similar to Reference [15], the DSE can be realized by the basic neural networks named MLP, where the inputs are the objectives and the **network parameters** that interpret the neural network architectures, and the outputs are the configurations. The training flow of MLP is shown in Figure 3(a). First, the training set is generated by evenly choosing enough sets of the network parameters and

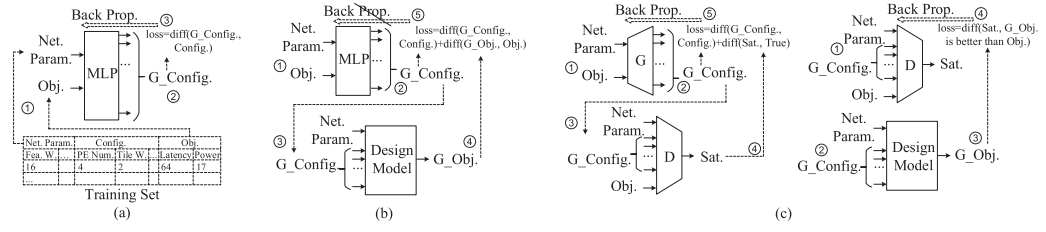


Fig. 3. Training flows in different DSE schemes: (a) barely using MLP; (b) using MLP with the design model for improving the loss function, which is not viable; (c) using GAN, where the left and right parts are for training the G and the D, respectively. (“Net. Param.,” “Obj.,” “G_Config.,” and “G_Obj.” stand for network parameters, objectives, generated configurations, and objectives under the generated configurations, respectively.)

configurations, and using the design model for the corresponding objectives. Then, by using the difference between the configurations in the training set and the generated configurations as the loss, the MLP is trained to learn the distribution of the data in the training set. Ideally, the MLP is able to generate the satisfied configurations during the inference.

However, there is a critical challenge in the approach in Figure 3(a). That is, if the generated configurations differ from the corresponding configurations in the training set, then it is regarded as a wrong set of configurations, since it incurs the loss. Nevertheless, there are usually multiple sets of the configurations that can satisfy the objectives for the given network parameters. Although the generated configurations are different from those in the training set, it is still possible that the generated configurations are also satisfied, or even better satisfied the objectives. The naive approach in Figure 3(a) ignores this scenario, so it can miss many opportunities to generate better configurations.

To try to address this problem, a direct approach is to use the design model to verify if the generated configurations satisfy the objectives, when the generated configurations are different from the training set. The training flow is in Figure 3(b). However, the loss function contains the corresponding objectives “G_Obj” by the generated configurations, which relates to the weights of the MLP and the design model. This means the gradients of MLP also relate to the design model. As the design model is not necessarily analytic, it means the loss is not necessarily able to be back propagated. Therefore, the approach in Figure 3(b) is not viable.

Finally, inspired by using the GAN (specifically, cGAN) for the text-image problem, where the scenario is similar (creating the data that satisfy the objectives), we proposed a GAN-based DSE algorithm. The training flow of the proposed GAN is shown in Figure 3(c). Compared with Figure 3(b), now the MLP is called the generator (G), and the design model is replaced by the discriminator (D). The D outputs if the configurations can satisfy the objectives given the network parameters, which is called “satisfaction” (“Sat.” in Figure 3(c)). In this case, since the calculations in both G and D are differentiable, the loss can be back propagated to train G. When training D, the generated configurations from G are input to D to get the satisfaction prediction. Besides, the corresponding objectives “G_Obj” of the same generated configurations are obtained from the design model. If “G_Obj” is overall better than the user’s objectives, then the actual satisfaction should be “True”; otherwise, it is “False.” Then the actual satisfaction is compared with the satisfaction prediction to get the loss. Note that although the actual satisfaction in the loss function relates to the design model, it does not relate to the weight in D. This means, the actual satisfaction is a constant for calculating the gradients of D, and thus, the same problem in Figure 3(b) does not exist in this case. The G and D are trained alternatively based on each other’s outputs, and thus the adversarial training is realized.

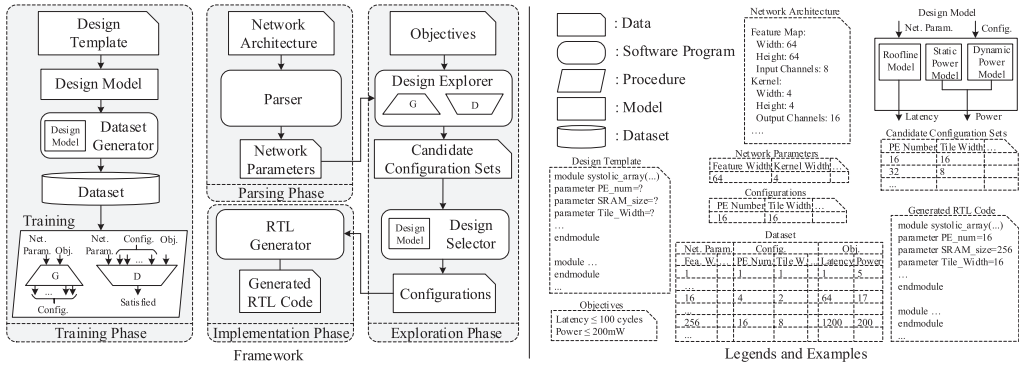


Fig. 4. Overview of GANDSE framework. (The training phase only needs to be performed once for each design template. Other three phases need to be performed for each set of network architectures and objectives.)

5 SYSTEM OVERVIEW

Based on the discussion in Section 4, an end-to-end neural network accelerator design automation framework called GANDSE is proposed to search for the highly optimized design in a high-dimension large design space with small time cost, where a novel DSE flow is designed. GANDSE can generate the optimized neural network accelerator, which can be used for the inference of the given neural networks while satisfying the user's objectives of the latency and the power consumption. The framework of GANDSE is shown in Figure 4, with three inputs and four phases involved.

The inputs are listed in the following.

- **Design Template:** The neural network accelerator design template with a set of configurations needed to be determined, which is written in synthesizable RTL code. The configurations not only contain the **architecture parameters** such as the SRAM size but also contain the **mapping strategies** such as the tiling size of the neural networks.
- **Network Architecture:** The layer architecture of the neural networks that need to be executed on the generated neural network accelerator design. An example is shown in Figure 4.
- **Objectives:** The user's requirements to the generated neural network accelerator when performing the inference of the given network architecture. Our work uses latency and power as the examples, but the objectives are not limited to them. Besides, the format of the objectives in GANDSE is "objective $\leq x$," where x is a number given from the user.

The four phases are briefly described in the following.

- **Training Phase:** The design explorer in GANDSE is trained to be able to generate the optimized configurations of the template. For each design template, the training phase only needs to be performed for one time.
- **Parsing Phase:** This phase parses user readable inputs to the numbers that can be directly used in the next phase.
- **Exploration Phase:** This phase generates the optimized configurations, which satisfies the user's objectives.
- **Implementation Phase:** This phase applies the generated configurations to the design template and generates the synthesizable RTL code of the neural network accelerator.

After performing the training phase to set up GANDSE according to the given neural network accelerator design template, GANDSE can automatically generate synthesizable RTL code of the

neural network accelerator with the configurations, which satisfies the user objectives when executing the neural networks with the given network architecture. The circuits synthesized from the generated RTL code can be implemented on an FPGA.

The details of the four phases are introduced as follows.

5.1 Training Phase

Before GANDSE is able to be used, the critical component, which is the GAN in the **Design Explorer**, needs to be trained as the flow in Figure 3(c). Note that different from the GNN that is used for the design model in Reference [18], the GAN in the design explorer is for creating new configurations according to the given the objectives, which is the reverse of the design model, and has different challenges discussed in Section 4.

Given the design template, the design model is created to predict the latency and power. The latency is constructed by using the roofline model, while the power is constructed by combining the static power model and the dynamic power model of the template. The design model is also verified by simulation and synthesis.

After the design model is constructed, an automatic **Dataset Generator** is employed to generate a dataset containing various neural network architectures, configurations, and the latency and power consumption. The number of the data in the dataset is large enough so that the data can evenly cover the design space. An example of the generated dataset is shown in Figure 4.

Next, the GAN inside the design explorer is trained with the dataset, and then applied to the design explorer. Since the generator of the GAN in the design explorer learns the mapping from the network parameters and objectives to the configurations through the adversarial learning, it is able to generate the configurations given the user's objectives in the exploration phase.

5.2 Parsing Phase

In the parsing phase, the **Network Parser** can parse the user's description that contains the information of the neural network architectures, then generate the **network parameters** that can be directly input to the next phase along with the objectives.

The user's description is an abstract intermediate representation for the neural network architectures, such as the network architecture example in Figure 4. It can be handwritten or extracted from deep learning frameworks such as Caffe [8] or PyTorch [14].

5.3 Exploration Phase

With the outputs of the parsing phase, the exploration phase can generate the optimized configurations of the neural network accelerator design template that satisfy the user's latency and power objectives. The **Design Explorer** contains a GAN, which is trained in the offline phase. Based on the generator networks in the GAN, the design explorer tries to generate multiple sets of the configurations including the architecture parameters and the mapping strategies, so that when the configured neural network accelerators are implemented on the FPGA to execute the neural networks with the given network parameters, the latency and power objectives can be satisfied. These sets of the configurations are called **Candidate Configuration Sets**. After the candidate configuration sets are generated, the **Design Selector** then selects the most optimized set of configurations with the use of the selection algorithm, which will be introduced in Section 6.

5.4 Implementation Phase

Using the design template with the optimized configurations including the architecture parameters and the mapping strategies, the **RTL Generator** can generate the synthesizable RTL code, which can be implemented on FPGA boards directly.

ALGORITHM 1: The proposed training scheme. (Net_s : Network parameters of sample s ; $Config_s$: Configurations of s ; LO_s : Latency objective of s ; PO_s : Power objective of s .)

Require: G , generator. D , discriminator. Set_{train} , training set. $iter$, number of iterations. M_l , latency model. M_p , power model. E , cross entropy loss function. bs , batch size.

```

1: for  $iter$  of training iterations do
2:   for Each batch  $b \in Set_{train}$  do
3:      $Loss_{config}, Loss_{critic}, Loss_{dis} \leftarrow 0$ 
4:     for Each sample  $s \in b$  do
5:        $Config_g \leftarrow G(Net_s, LO_s, PO_s)$ 
6:        $Sat \leftarrow D(Net_s, Config_g, LO_s, PO_s)$ 
7:        $L_g \leftarrow M_l(N_s, Config_g)$ 
8:        $P_g \leftarrow M_p(N_s, Config_g, L_g)$ 
9:        $Loss_{critic} + = E(Sat, True)/bs$ 
10:      if  $L_g \leq LO_s$  and  $P_g \leq PO_s$  then
11:         $Loss_{config} + = 0$ 
12:         $Loss_{dis} + = E(Sat, True)/bs$ 
13:      else
14:         $Loss_{config} + = E(Config_s, Config_g)/bs$ 
15:         $Loss_{dis} + = E(Sat, False)/bs$ 
16:      end if
17:    end for
18:    update  $G$  with  $Loss_{config} + w_{critic} \times Loss_{critic}$ 
19:    update  $D$  with  $Loss_{dis}$ 
20:  end for
21: end for
22: return  $G$  and  $D$ 

```

6 GAN-BASED DESIGN SPACE EXPLORATION

In this section, the critical part of GANDSE, which is the GAN-based DSE algorithm, is further elaborated. The DSE algorithm relates to the training of the GAN, the design explorer (the inference of the GAN), and the design selector in Figure 4.

6.1 Training and Inference Schemes for GAN-based DSE

The details of the training flow proposed in Section 4 are further introduced. The proposed training scheme is shown in Algorithm 1, during which the **generator (G)** and the **discriminator (D)** are trained alternately. The expectation is, G can output the configurations that can satisfy the objectives for given the network parameters, when the configured neural network accelerator is implemented on the FPGA. In contrast, the D takes network parameters, configurations, and user's objectives as the inputs. The D will decide if the objectives can be satisfied or not and outputs the satisfaction as "True" or "False," respectively. Note that if and only if all the objectives are better than the user's requirements, the satisfaction is "True." The configurations are one-hot encoded, since most of the configurations of the architectures and mapping strategies are not successive and only some specific numbers are meaningful. Otherwise, the generated configurations might be decimal or negative, which can not be employed. The user's objectives and the network parameters are encoded as the binary numbers, and normalized by the standard deviation. We also employ the one-hot encoding for the satisfaction output of D, similar to other neural networks classification tasks.

The detailed training procedures are illustrated in Algorithm 1, where we use the mini-batch training (Lines 1–4). For each sample s in the training set, the user's objectives LO_s and PO_s and

the network parameters Net_s are input to G to get the generated configurations $Config_g$ (Line 5). The generated configurations are then input to D to get the satisfaction Sat (Line 6). Next, $Config_g$ are input to the latency model M_l and power model M_p to get the actual latency L_g and power consumption P_g (Lines 7 and 8), where M_p may need the latency L_g as input of the dynamic power model. Then, three losses need to be calculated. The configurations loss $Loss_{config}$ represents if the generated configurations $Config_g$ are close to the actual configurations $Config_s$ of sample s . The critic loss $Loss_{critic}$ represents if D believes the generated configurations $Config_g$ satisfies the user's objectives or not. The discriminator loss $Loss_{dis}$ represents if the classification on the satisfaction made by D is correct or not. $Loss_{critic}$ is calculated by Line 9 by cross entropy loss. The actual latency L_g and power P_g corresponding to the generated configurations are then compared with the objectives LO_s and PO_s of the sample s (Lines 10–16). If L_g and P_g are both better than LO_s and PO_s , respectively, then $Config_g$ is regarded as the satisfied configurations, so there is no $Loss_{config}$ (Line 11), and the output Sat of D should be “True” (Line 12). Otherwise, $Loss_{config}$ is calculated by Line 14, and Sat needs to be false (Line 15). After calculating the losses for one batch of the samples, G and D are backpropagated and updated (Lines 18 and 19). Note that we employ the weight w_{critic} for $Loss_{critic}$ to control the effect of D.

The trained G and D are used by the design explorer. When performing the inference, given the network parameters and the objectives, G can output the configurations encoded by one-hot. Since ordinary one-hot encoding outputs the probabilities of each choice of each configuration, we use another number between 0 and 1 called **Probability Threshold** (such as 0.2), to allow multiple sets of generated configurations output from G, which are the candidate configuration sets shown in Figure 4. For each configuration, if the one-hot output of one choice exceeds the probability threshold, then the choice is employed. Then the candidate configuration sets are the combinations of all the employed choices of all the configurations. For example, assume there are only two kinds of configurations that are the PE number and the SRAM size, the configurations are denoted as (PE number, SRAM size). If the one-hot outputs of “PE number = 4,” “PE number = 16,” “SRAM size = 2 KB,” and “SRAM size = 8 KB” are more than 0.2, then there are four sets of configurations that are candidates, which are (4, 2 KB), (4, 8 KB), (16, 2 KB), and (16, 8 KB).

6.2 Selection Algorithm

Since normally there are thousands of the candidate configuration sets after performing the inference on G in the design explorer, the design selector, which is a program, is proposed to select the most optimized set of the configurations.

In Algorithm 2, we keep updating two variables L_{opt} and P_{opt} recording the optimized latency and power objectives (Lines 1 and 2), respectively. For each set of the configurations $Config_g$ in the candidate configurations sets (Line 3), the corresponding actual latency L_g and power P_g are calculated by the design model (Lines 4 and 5). The algorithm maintains a variable $Update$ to indicate if L_{opt} and P_{opt} need to be updated (Line 6). For example, if L_{opt} and P_{opt} have never been updated, they need to be initialized for the first time (Lines 7 and 8). Otherwise, there are three scenarios to perform the updating. When both current optimized objectives (L_{opt} and P_{opt}) are worse/better than the user's objectives (LO and PO), it is the first scenario. In this scenario, if the objectives (L_g and P_g) of $Config_g$ this time are both better than L_{opt} and P_{opt} , respectively, L_{opt} and P_{opt} need to be updated (Lines 10–13). This can ensure that the optimized objectives (L_{opt} and P_{opt}) keep changing for the better. In contrast, The second scenario is that the current optimized latency L_{opt} is worse than the user's objective LO , while the current optimized power P_{opt} satisfies the objective PO . In this case, if the latency L_g of $Config_g$ this time is better than the current optimized latency L_{opt} , and the power P_g still satisfies the user's objective PO , even though P_g might be worse than the current optimized power P_{opt} , the updating is still performed (Lines 15–18). This is

ALGORITHM 2: The proposed selection algorithm of the design selector.

Require: Set_{config} , candidate configuration sets. M_l , latency model. M_p , Power model. Net , the network parameters from the user. LO , the user's latency objective. PO , the user's power objective.

```

1:  $L_{opt} \leftarrow 0$ 
2:  $P_{opt} \leftarrow 0$ 
3: for Each  $Config_g \in Set_{config}$  do
4:    $L_g \leftarrow M_l(Net, Config_g)$ 
5:    $P_g \leftarrow M_p(Net, Config_g)$ 
6:    $Update \leftarrow False$ 
7:   if  $L_{opt} == 0$  and  $P_{opt} == 0$  then
8:      $Update \leftarrow True$ 
9:   else
10:    if ( $L_{opt} > LO$  and  $P_{opt} > PO$ ) or ( $L_{opt} < LO$  and  $P_{opt} < PO$ ) then
11:      if  $L_g < L_{opt}$  and  $P_g < P_{opt}$  then
12:         $Update \leftarrow True$ 
13:      end if
14:    else
15:      if  $L_{opt} > LO$  and  $P_{opt} < PO$  then
16:        if  $L_g < L_{opt}$  and  $P_{opt} < PO$  then
17:           $Update \leftarrow True$ 
18:        end if
19:      else
20:        if  $P_g < P_{opt}$  and  $L_{opt} < LO$  then
21:           $Update \leftarrow True$ 
22:        end if
23:      end if
24:    end if
25:  end if
26:  if  $Update$  then
27:     $L_{opt} \leftarrow L_g$ 
28:     $P_{opt} \leftarrow P_g$ 
29:     $Config_{opt} \leftarrow Config_g$ 
30:  end if
31: end for
32: return  $Config_{opt}$ 

```

because for the second scenario, the most significant priority is to make all the objectives satisfied. The third scenario (Lines 20–22), where the conditions of the latency and power are reversed, is handled similarly to the second scenario. Finally, the optimized configurations $Config_{opt}$ that have the most optimized latency L_{opt} and power P_{opt} are returned.

7 EXPERIMENTS

7.1 Experimental Setup

The experiments are conducted on Intel Xeon Silver 4210R CPU @ 2.40 GHz with 64 GB DDR4 memory, and NVIDIA GeForce RTX 3090. The synthesis tool is Vivado 2018.3. The GAN of the framework is deployed and trained using PyTorch [14].

7.1.1 Design Model. In our experiments, CNNs are used as the neural networks that will be executed on the neural network accelerators to be generated, since CNN is one of the most popular neural networks. There are two design models of the neural network accelerators involved in our

Table 1. Network Parameters, Configurations, and Objectives Used in Our Experiments

Network Parameters (CNN)	Input Channel, Output Channel, Output Width, Output Height, Kernel Width, Kernel Height
Configurations: Architecture Parameters	PE Number, SRAM to DRAM Bandwidth*, DRAM to SRAM Bandwidth*, Input SRAM Size, Weight SRAM Size, Output SRAM Size
Configurations: Mapping Strategies	Tiling Input Channel*, Tiling Output Channel*, Tiling Output Width*, Tiling Output Height*, Tiling Kernel Width*, Tiling Kernel Height*
Objectives	Latency, Power

*The configuration is not included in DnnWeaver model.

experiments, where the data-flow is output stationary, but other design models can also be applied to GANDSE.

im2col model: This design model adopts the *im2col* computation pattern [2] similar to GPU computation. As shown in Table 1, this model explores the limits of design space and includes configurable parameters as many as possible. The establishment of this model is based on the roofline model, which analyzes the relationship between DRAM bandwidth, SRAM bandwidth, and on-chip computation. This model is used to illustrate the effectiveness of GANDSE in high-dimension design space, and can represent the general behaviors of the hardware architectures with im2col computation patterns. Therefore, this model does not correspond to a specific RTL design template in our experiments.

DnnWeaver model: Based on the open-source DnnWeaver code [5] of the systolic array as the design template, the DnnWeaver model is established. The model is calibrated by the simulation and synthesis results of code to ensure the output of the model is aligned with the metrics of hardware employment. After the configurations are found by GANDSE, they can be written as parameters in the RTL code of DnnWeaver. Note that the number of the configurations for the DnnWeaver model is larger than that in the paper of DnnWeaver, since we extend the allowed configurations of the DnnWeaver code. This model is used to make a comparison between GANDSE and DnnWeaver.

Table 1 shows the detailed network parameters, configurations, and objectives of the design models in our experiments. Note that the latency is the end-to-end latency, which considers three pipelined phases for each tile, including loading data, computing, and writing back the data. Besides, the objectives need to be set as the upper bound that the user can accept. For example, when the user's latency and power objectives are set as LO and PO , respectively, GANDSE considers that the user needs latency $\leq LO$ and power $\leq PO$.

7.1.2 Dataset. The dataset is generated by Dataset Generator discussed in Section 5.1, where the values of the network parameters, the architecture parameters, and the mapping strategies are evenly sampled to cover most of the scenarios. In this case, various CNN layers can be learned. Then, the objectives are calculated by the design model. For im2col model, the dataset contains 23,420 data for the training and 1,000 for the testing. For DnnWeaver model, the dataset contains 31,250 data for the training and 1,000 for the testing. Note that GANDSE does not restrict the design model, network parameters, configurations, and objectives, so other parameters (e.g., stride, padding, etc.) can also be supported with corresponding design models. When generating the dataset, with the number of the parameters and configurations increasing, the sample rate can be reduced for the trade-off between the training time and the number of scenarios covered. Besides,

Table 2. Example Part of the Dataset of im2col Model

IC	OC	OW	OH	KW	KH	PEN	SDB	DSB	ISS	WSS	OSS	TIC	TOC	TOW	TOH	TKW	TKH	L	P
32	32	32	32	1	1	2,048	32	64	512	512	512	64	16	16	128	4	1	0.0014	0.1098
32	32	32	32	1	1	1,024	32	128	512	512	512	16	64	16	16	5	5	0.0018	0.0977
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
64	32	64	64	5	5	512	128	128	4,096	2,048	4,096	64	64	16	256	5	1	0.0551	2.4652
64	32	64	64	5	5	2,048	128	512	4,096	2,048	4,096	16	64	64	64	1	1	0.0137	3.9144
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

The abbreviations of the columns are the combinations of the first letters of the names in Table 1.

Table 3. Example Part of the Dataset of DnnWeaver Model

IC	OC	OW	OH	KW	KH	PEN	ISS	WSS	OSS	L	P
16	16	16	16	3	3	8	128	128	128	0.0171	1.0237
16	16	16	16	3	3	16	128	128	128	0.0144	1.2063
...	...	...	...	...	...	...	...	...	...	...	...
16	64	32	32	3	3	16	512	2,048	512	0.2462	1.2201
16	64	32	32	3	3	32	512	2,048	512	0.2463	1.2157
...	...	...	...	...	...	...	...	...	...	...	...

The abbreviations of the columns are the combinations of the first letters of the names in Table 1.

Table 4. Hyperparameters of the GAN

Design Model	G					D				
	Hidden Layers	Neurons Per Layer	Learning Rate	Activation Function	Optimizer	Hidden Layers	Neurons Per Layer	Learning Rate	Activation Function	Optimizer
im2col	11	2,048	2e-5	ReLU	Adam	11	2048	2e-5	ReLU	Adam
DnnWeaver	14	2,048	2.5e-5	ReLU	Adam	11	2048	2.5e-5	ReLU	Adam

w_{critic} is not listed as it is separately discussed in the results; The batch sizes under both design models are 1,024.

GANDSE does not target a specific CNN architecture, and various architectures generated in the dataset will be tested. The example parts of the dataset are shown in Tables 2 and 3, where the latency and power are normalized by the standard deviation.

7.1.3 Hyperparameters and Training. Table 4 listed the hyperparameters when training the proposed GAN in GANDSE, where both G and D are multilayer perceptrons. The training phase only needs to be performed for each design template. In our experiments, we only tune the number of the layers, the number of the neurons in each layer (which is the same in each layer, and is tuned by setting it as 512/1,024/2,048/4,096), and the learning rate (which is tuned by setting it in the range of 1e-5 to 1e-4). Therefore, simple hyperparameter tuning is already enough to make GANDSE have good DSE results after training.

7.1.4 Compared DSE Algorithms. The following DSE algorithms are used for both im2col model and DnnWeaver model for comparison.

- **Simulated annealing (SA).** Simulated annealing is used for comparison. SA terminates once the user's objectives are satisfied, or the temperature is 3×10^{-8} as the initial one.
- **Deep reinforcement learning (DRL).** For DRL, similar to ConfuciusX [9], the policy gradient is employed in our experiment, and a neural network is used as the actor network to update the underlying policy network. The states are the current network parameters and configurations, and the actions are the modifications to the configurations. The reward is obtained when the current action is approaching the states that satisfied the objectives. When the current state already satisfies the objectives, a bonus is also added to the reward.

Table 5. DSE Results of Different Methods under Two Design Models

	Method	w_{critic}	Training Time	# of Cand. Config. [†]	# of NN Param. [‡]	DSE Time	# of Sat. Results	Improvement Ratio
im2col (Large)	SA	—	—	—	—	17.60 s	786/1,000	0.2526
	DRL	—	133,709 s	—	93M	0.08 s	687/1,000	0.3399
	Large MLP	—	47,267 s	10,794.01	93M	1.11 s	839/1,000	0.3281
	GAN	0	78,806 s	732.68	93M	0.09 s	554/1,000	0.3286
		0.5	116,229 s	5,360.19	93M	0.59 s	944/1,000	0.3579
		0.7	118,474 s	5,059.44	93M	0.74 s	792/1,000	0.3405
		1	116,228 s	6,861.21	93M	0.77 s	893/1,000	0.3458
DnnWeaver Model (Small)	SA	—	—	—	—	13.60 s	936/1,000	0.0010
	DRL	—	27,634 s	—	110M	<0.01 s	948/1,000	0.0019
	Large MLP	—	11,618 s	26.54	110M	<0.01 s	862/1,000	0.0015
	GAN	0	173,345 s	20.17	105M	<0.01 s	847/1,000	0.0017
		0.5	36,345 s	21.17	105M	<0.01 s	938/1,000	0.0014
		1.0	36,298 s	18.42	105M	<0.01 s	964/1,000	0.0015
		1.2	36,421 s	20.52	105M	<0.01 s	945/1,000	0.0016
		1.5	25,224 s	14.2	105M	<0.01 s	938/1,000	0.0016

[†]“Cand. Config.” stands for candidate configurations; [‡]“Param.” stands for parameters; NN is the neural network used for DSE but not the one to be executed on the accelerator.

After performing hyperparameter tuning, the relatively better results are shown in our experiments.

- **Large multilayer perceptron (Large MLP).** Similar as AIRCHITECT [15], we also tested the results when only applying the MLP as shown in Figure 3(a). Besides, we also apply the design selector to improve the results. To show the effectiveness of GANDSE, the number of the parameters in the MLP is set to match that in the GAN, which makes the MLP much larger than the G in the GAN. The training epochs of both GAN and large MLP are also set to the same.

All the compared DSE algorithms are modified to perform DSE based on the same system-level architectures (design templates) as GANDSE for fair comparison.

7.2 GANDSE Results Analysis

We first analyze the effectiveness of the GAN-based DSE algorithm in GANDSE. The results are shown in Table 5, where the bold numbers are the main results. When training GAN, different values of w_{critic} in Algorithm 1 are chosen. Note that we allow 1% of the noise when evaluating all DSE algorithms in Table 5, which means if the latency/power of the DSE output is $\leq 1\%$ worse than the user’s objective, the objective is still regarded as satisfied.

Under im2col model, different w_{critic} values have different effects on the GAN-based DSE algorithm. When $w_{critic} = 0$, the D has no effect in the GAN, and the training scheme is the same as Figure 3(a) with MLP only. Given 1000 DSE tasks, there are only around 50% results that satisfy the user’s objectives. In contrast, when $w_{critic} = 0.5$, the D can help the learning of G, and almost all the results successfully satisfy the objectives. When further increasing w_{critic} , the results start getting slightly worse but still much better than that without D. This is because the satisfaction classification result from D contributes too much to the loss of G. It becomes hard for G to directly learn from the training set. The results show the effectiveness of GANDSE for finding the optimized configurations, and by hyperparameter tuning, GANDSE can be further improved with a proper w_{critic} (0.5 under im2col model).

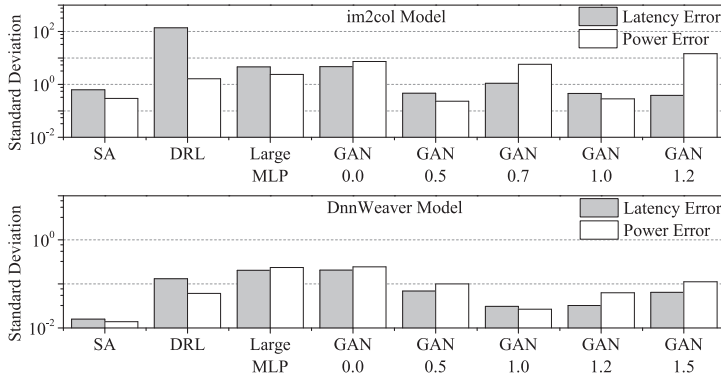


Fig. 5. Standard deviations of latency and power errors. (The numbers under “GAN” are w_{critic} .)

Table 5 shows the average runtime of each DSE task. The GAN-based DSE algorithm with $w_{critic} = 0.5$ only needs < 1 s to complete one DSE task, which is negligible in the hardware design flow. Besides, the value of w_{critic} can affect the runtime. This is mainly because of the different numbers of candidate configurations.

Besides, the improvement ratio is also analyzed. GANDSE can find the configurations that outperform the user’s objectives. Assume the latency and power objectives of the configurations from GANDSE are L_{opt} and P_{opt} , respectively, and the user’s objectives are LO and PO , the improvement ratio is defined as $\sqrt{\frac{1}{2} \times ((\frac{L_{opt}-LO}{LO})^2 + (\frac{P_{opt}-PO}{PO})^2)}$ when $L_{opt} \leq LO$ and $P_{opt} \leq PO$. This can indicate how much the results of GANDSE outperform the objectives. When one result is not satisfied, there is at least one objective (latency or power) worse than the user’s objectives, so the result is invalid and there is no improvement ratio for this result. Shown in Table 5, with a proper w_{critic} , the results of GANDSE are more than 35% better than the user’s objectives on average. This also proves the effectiveness of GANDSE.

The training time of GANDSE is more than large MLP, but it is still acceptable. Once it is trained, it can be used for any times for high-quality designs given different objectives. Meanwhile, compared with the time period needed by the hardware design, which are counted by months, the training time is already negligible.

Figure 5 shows the standard deviations of latency and power errors. The latency and power errors are defined as $\frac{L_{opt}-LO}{LO}$ and $\frac{P_{opt}-PO}{PO}$, respectively. It is indicated that for im2col model, GAN-based DSE has the least standard deviation when $w_{critic} = 0.5$. This also proves that D can help the regression of the results of G, compared training G only when $w_{critic} = 0$.

We also test the effectiveness of GANDSE on DnnWeaver model, which has a much lower dimension of design space. For DnnWeaver model, the optimized design can be obtained by training the GAN with $w_{critic} = 1.0$ in Table 5. If $w_{critic} = 0$ and the training of G is not affected by the D, compared with the results of $w_{critic} = 1.0$, then fewer results satisfy the user’s objectives. Meanwhile, when w_{critic} is too large, the results are also slightly worse than $w_{critic} = 1.0$ but are still much better than $w_{critic} = 0$. Besides, shown in Figure 5, when training GAN, the setup with the best results also has small enough standard deviations for latency and power errors, proving the effectiveness of the regression. Although SA has a smaller standard deviation, the number of the satisfied results and the improvement ratio are both worse than GAN. More importantly, compared with GAN, SA performs worse in all metrics under the large design model. The average runtime under DnnWeaver model is around 0.01 s, which is negligible. The improvement ratio of GAN on DnnWeaver model is not as obvious as large im2col model, but most of the results

still satisfy the user's objectives. Overall, the results under DnnWeaver model also indicate the effectiveness of the proposed DSE algorithm and the automation framework, and GANDSE has more advantages on the DSE within high-dimension large design space.

7.3 Comparison with Other DSE Algorithms

We also compared GANDSE with other DSE algorithms including SA, DRL, and large MLP. The results are shown in Table 5 and Figure 5. For Large MLP, although the parameter size of this MLP is the same as or even more than that of the GAN in GANDSE, the large MLP performs worse compared with GAN. First, large MLP finds almost 100 fewer satisfied results under both models, compared with GANDSE. One can notice that MLP has a much larger candidate configuration set size, which implies that MLP has less confidence in the results. Next, large MLP also have a worse improvement ratio. Large MLP have almost $1\times$ more runtime under im2col model. Finally, the standard deviation of the MLP results shown in Figure 5 is also much larger than that of GANDSE's results. This proves that the effectiveness of GAN-based DSE cannot be achieved by simply increasing the size of a MLP to the same level, without the guidance of D network. For DRL, after performing hyperparameter tuning, the relatively better DRL results are obtained in our experiments. Although DRL also has smaller runtime, it has worse results and standard deviations under both im2col and DnnWeaver model, compared with the GAN-based DSE. Under DnnWeaver model, DRL has a slightly better improvement ratio, but its improvement ratio under im2col model is still worse than GAN-based DSE. For SA, it performs better under the small model than the large model, but the detailed number of satisfied results, the standard deviation of the results, the runtime, and the improvement ratio are all worse than GAN-based DSE. In conclusion, GAN-based DSE has overall better performance in the comparison.

7.4 DSE Results with Different Objective Difficulties

We also analyze the effectiveness of GANDSE given the user's objectives with different difficulties. First, the difficulty is quantified as follows. There are multiple Pareto frontiers in the dataset. (For a sample s , if there is no other sample with one objective better than s and other objectives not worse than s , s is a Pareto frontier). The difficulty of the user's latency and power objectives (LO, PO) is quantified by using the Euclidean distance between points (LO, PO) and (L_{pareto}, P_{pareto}) , which is the closest Pareto frontier to (LO, PO) . Besides, the Euclidean distance is then normalized by the module of (L_{pareto}, P_{pareto}) . Then, user's objectives with smaller distances to the closest Pareto frontiers have higher difficulties to be satisfied. Figures 6 and 7 indicate the percentage of the satisfied results (y-axis) given the topmost $n\%$ difficult user's objectives (x-axis). One can easily figure out that most DSE algorithms have better results with more objectives that are easier to be satisfied. Under im2col model (Figure 6), when selecting the topmost 10% difficult objectives (including the Pareto frontiers), only GAN ($w_{critic} = 0.5$ and 1.0), Large MLP, and SA has relatively better results. However, with the growing size of the objectives, GAN ($w_{critic} = 0.5$ and 1.0) has a better ratio of the satisfied results, while the curves of other approaches fluctuate and become worse than GAN. Since GAN with $w_{critic} = 0$ has no D, it is not a real GAN structure and the results are worse than those with D involved. Note that with objective difficulty decreasing, Large MLP has even worse results compared with difficult objectives. This might be because it strictly learns the architectures corresponding to the objectives in the training set, and misses the opportunities to learn better architectures, which has been discussed in Section 4. This is even worse for easy objectives, since there are more such opportunities. Similar to im2col model, under DnnWeaver model (Figure 7), the results also prove the effectiveness of GAN, even with difficult objectives. In contrast, other approaches perform worse when the objectives are difficult to be satisfied.

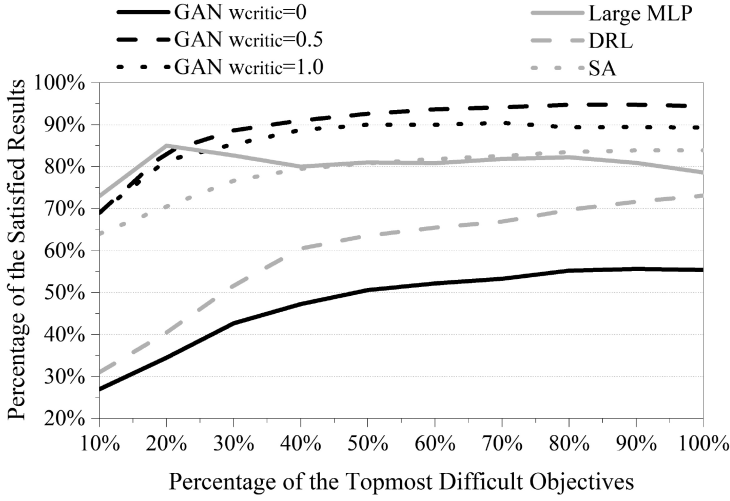


Fig. 6. Percentage of the satisfied results of the objectives with different difficulties under im2col model.

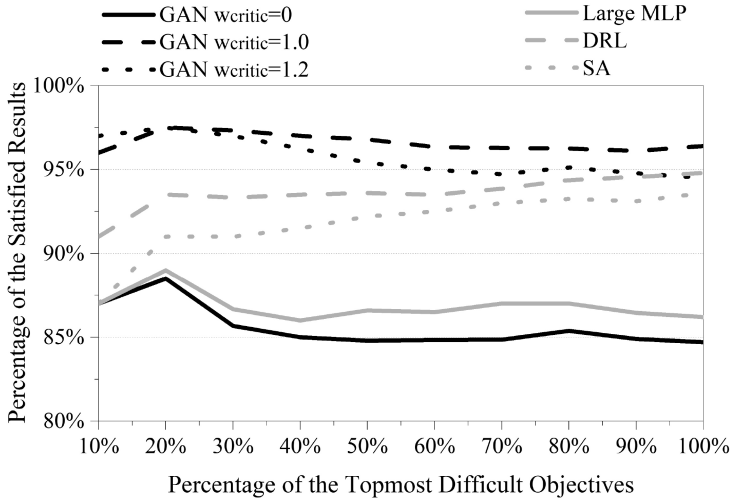


Fig. 7. Percentage of the satisfied results of the objectives with different difficulties under DnnWeaver model.

7.5 Results Distribution Analysis

In this subsection, we analyze the result distribution of different DSE algorithms, which is shown in Figures 8 and 9. Each dot is a result of the DSE, which indicates the improvements of the latency/power of the DSE output configurations compared with the user's objectives. The x-axis and y-axis represent $\log_2(\frac{LO}{L_{opt}})$ and $\log_2(\frac{PO}{P_{opt}})$, respectively. For Figure 8(a), under im2col model, when $w_{critic} = 0$, the D has no effect in the GAN. One can easily figure out that there are lots of results in the second (or fourth) quadrant, which represent they fail to satisfy the user's latency (or power) objective. In contrast, when $w_{critic} = 0.5$ in Figure 8(b), the D can help the learning of G, and the results are well regressed into the first quadrant. However, with the increasing of w_{critic} to 1.0 in Figure 8(c), the results are getting slightly worse. This is because it becomes harder for G to directly learn from the training set than from D.

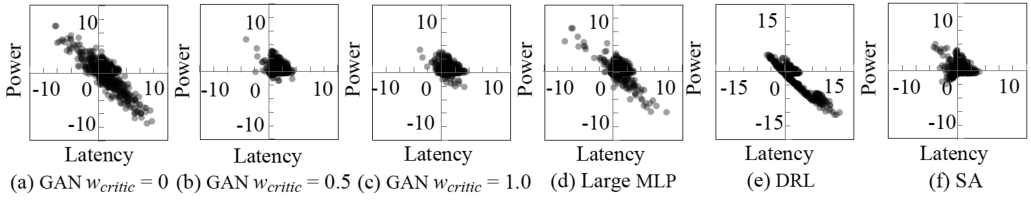


Fig. 8. Latency and power improvement results (dots) of the different DSE algorithms based on im2col model. A dot with non-negative coordinate of an axis represents the satisfaction of the corresponding objective.

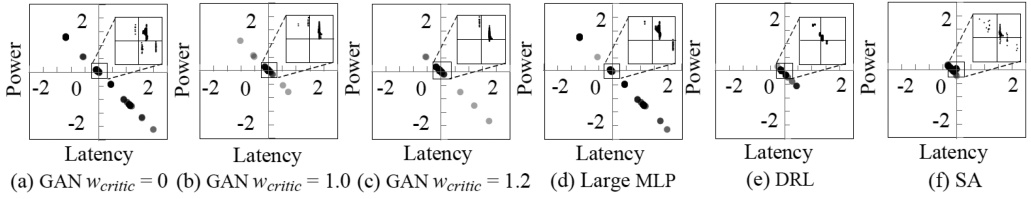


Fig. 9. Latency and power improvement results (dots) of the different DSE algorithms based on DnnWeaver model. A dot with non-negative coordinate of an axis represents the satisfaction of the corresponding objective.

For Figure 9, as most results are located near the origin (0,0), this part is enlarged inside the square at the top-right of each subfigure. For Figures 9(a)–9(c), since the design space has a much smaller dimension, the results distribution under DnnWeaver model is different from that under im2col model, but more results are still in the first quadrant with the effect of D than that without D ($w_{critic} = 0$), enlarged in the square at the top right. The best w_{critic} can be obtained from hyperparameter tuning.

For im2col model, compared with Figure 8(a), large MLP in Figure 8(d) has similar distribution, but there are more dots successfully regress to the first quadrant due to larger number of parameters. However, for large MLP, there are still obvious distributions out of the first quadrant, which make the results worse than GAN-based DSE in Figures 8(b) and 8(c). This also proves the effectiveness of D even with fewer neural network parameters. For other previous approaches including DRL and SA, they both have worse distributions compared with GAN-based DSE, since there are more dots that fail to fall in the first quadrant. For DnnWeaver model, the related distributions are also similar to im2col model. In Figures 9(d) and 9(e), the density of the results in the second and fourth quadrants are obviously higher than that in Figure 9(b), which indicates fewer results are satisfied when using large MLP and DRL. For SA in Figure 9(f), although the results are concentrated to the middle, there are still fewer results in the first quadrant compared with GAN. The distribution comparison also shows that GAN can help GANDSE find better results.

7.6 Training Losses

More training insights can be analyzed by the losses recorded through the training, which are shown in Figures 10 and 11. Under im2col model, to analyze the G's output behaviors, when $w_{critic} = 0$, the D is still trained, but the output of D never affect the training of G. In this case shown in Figure 10(a), the critic loss of G keeps increasing, which indicates that D believes some sets of the configurations output from G cannot satisfy the user's objectives. When $w_{critic} = 0.5$ shown in Figure 10(b), since the classification result of D is used to train G, the critic loss of G is much lower than that in Figure 10(a). At the same time, all the losses become regressed, indicating

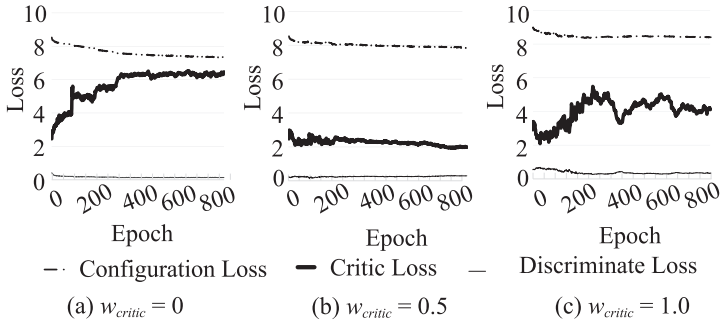


Fig. 10. Training losses of the GAN under im2col model.

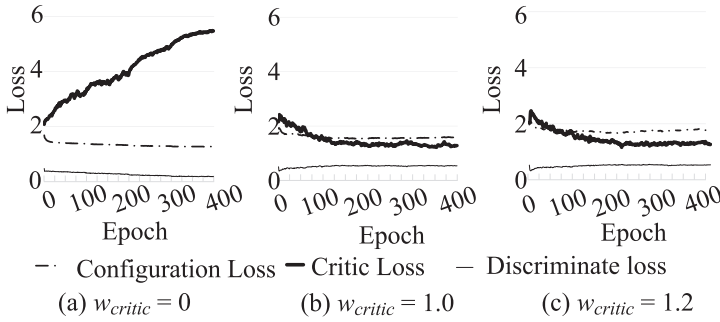


Fig. 11. Training losses of the GAN under DnnWeaver model.

the balance of G and D in the GAN. When $w_{critic} = 1.0$ shown in Figure 10(c), the critic loss cannot quickly regress as the classification results of D contribute too much and the G cannot easily learn from the training set. Similar conclusions can also be made from the training losses under DnnWeaver model shown in Figure 11. The results in Figures 10 and 11 show that different w_{critic} has different impacts on the training of the proposed DSE algorithm.

8 CONCLUSION

In conclusion, this work proposed a neural network accelerator design automation framework, GANDSE. With the neural network accelerator template that can be configured, given the user's objectives and the architecture of the neural networks that need to be executed on the neural network accelerator, GANDSE is able to effectively search the optimized configurations of the neural network accelerator that can satisfy the objectives. A novel DSE algorithm, which is based on GAN, is proposed and applied in GANDSE. Compared with the previous work, including using multilayer perceptron and deep reinforcement learning, the GAN-based DSE algorithm can effectively find more optimized designs within negligible time in the high-dimension large design space. The experiments proved the effectiveness of GANDSE.

REFERENCES

- [1] Kamel Abdelouahab, Cédric Bourrasset, Maxime Pelcat, François Berry, Jean-Charles Quinton, and Jocelyn Serot. 2016. A holistic approach for optimizing DSP block utilization of a CNN implementation on FPGA. In *Proceedings of the ACM International Conference on Distributed Smart Camera*. 69–75.
- [2] Kumar Chellapilla, Sidd Puri, and Patrice Simard. 2006. High Performance convolutional neural networks for document processing. In *Proceedings of the International Workshop on Frontiers in Handwriting Recognition*. 1–7.

- [3] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer, and Vivienne Sze. 2017. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE J. Solid-State Circ.* 52, 1 (2017), 127–138.
- [4] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient primitives for deep learning. Retrieved from <https://arxiv.org/abs/1410.0759>.
- [5] DnnWeaver v2.0. 2016. Retrieved from <http://dnnweaver.org/>.
- [6] Hasan Genc, Ameer Haj-Ali, Vighnesh Iyer, Alon Amid, Howard Mao, John Wright, Colin Schmidt, Jerry Zhao, Albert Ou, Max Banister, Yakun Sophia Shao, Borivoje Nikolic, Ion Stoica, and Krste Asanovic. 2019. Gemmini: An agile systolic array generator enabling systematic evaluations of deep-learning architectures. Retrieved from <https://arxiv.org/abs/1911.09925>.
- [7] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2020. Generative adversarial networks. *Commun. ACM* 63, 11 (2020), 139–144.
- [8] Yangqing Jia, Evan Shelhamer, Jeff Donahue, Sergey Karayev, Jonathan Long, Ross Girshick, Sergio Guadarrama, and Trevor Darrell. 2014. Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the ACM International Conference on Multimedia*. 675–678.
- [9] Sheng-Chun Kao, Geonhwa Jeong, and Tushar Krishna. 2020. ConfuciuX: Autonomous hardware resource assignment for DNN accelerators using reinforcement learning. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*. 622–636.
- [10] Yujun Lin, Mengtian Yang, and Song Han. 2021. NAAS: Neural accelerator architecture search. In *Proceedings of the ACM/IEEE Design Automation Conference*. 1051–1056.
- [11] Tao Luo, Shaoli Liu, Ling Li, Yuqing Wang, Shijin Zhang, Tianshi Chen, Zhiwei Xu, Olivier Temam, and Yunji Chen. 2017. DaDianNao: A neural network supercomputer. *IEEE Trans. Comput.* 66, 1 (2017), 73–88.
- [12] Mehdi Mirza and Simon Osindero. 2014. Conditional generative adversarial nets. Retrieved from <https://arxiv.org/abs/1411.1784>.
- [13] NVDLA. 2018. Retrieved from <http://nvdla.org/>.
- [14] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An imperative style, high-performance deep learning library. *Adv. Neural Info. Process. Syst.* 32 (2019), 1–12.
- [15] Ananda Samajdar, Jan Moritz Joseph, Matthew Denton, and Tushar Krishna. 2021. AIRCHITECT: Learning custom architecture design and mapping space. Retrieved from <https://arxiv.org/abs/2108.08295>.
- [16] Benjamin Carrion Schafer and Zi Wang. 2020. High-level synthesis design space exploration: Past, present, and future. *IEEE Trans. Comput.-Aided Design Integr. Circ. Syst.* 39, 10 (2020), 2628–2639.
- [17] Hardik Sharma, Jongse Park, Divya Mahajan, Emmanuel Amaro, Joon Kyung Kim, Chenkai Shao, Asit Mishra, and Hadi Esmaeilzadeh. 2016. From high-level deep neural models to FPGAs. In *Proceedings of the IEEE/ACM International Symposium on Microarchitecture*. 1–12.
- [18] Atefeh Sohrabizadeh, Yunsheng Bai, Yizhou Sun, and Jason Cong. 2022. Automated accelerator optimization aided by graph neural networks. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC'22)*. 55–60.
- [19] Atefeh Sohrabizadeh, Cody Hao Yu, Min Gao, and Jason Cong. 2022. AutoDSE: Enabling software programmers to design efficient FPGA accelerators. *ACM Trans. Design Autom. Electr. Syst.* 27, 4 (2022), 1–27.
- [20] Pengfei Xu, Xiaofan Zhang, Cong Hao, Yang Zhao, Yongan Zhang, Yue Wang, Chaojian Li, Zetong Guan, Deming Chen, and Yingyan Lin. 2020. AutoDNNchip: An automated DNN chip predictor and builder for both FPGAs and ASICs. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 40–50.
- [21] Hanqing Zeng, Ren Chen, Chi Zhang, and Viktor Prasanna. 2018. A framework for generating high throughput CNN implementations on FPGAs. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 117–126.
- [22] Xiaofan Zhang, Junsong Wang, Chao Zhu, Yonghua Lin, Jinjun Xiong, Wen-mei Hwu, and Deming Chen. 2018. DNNBuilder: An automated tool for building high-performance DNN hardware accelerators for FPGAs. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*. 1–8.
- [23] Xiaofan Zhang, Hanchen Ye, Junsong Wang, Yonghua Lin, Jinjun Xiong, Wen-mei Hwu, and Deming Chen. 2020. DNNExplorer: A framework for modeling and exploring a novel paradigm of FPGA-based DNN accelerator. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [24] Yongwei Zhao, Chang Liu, Zidong Du, Qi Guo, Xing Hu, Yimin Zhuang, Zhenxing Zhang, Xinkai Song, Wei Li, Xishan Zhang, Ling Li, Zhiwei Xu, and Tianshi Chen. 2021. Cambricon-Q: A hybrid architecture for efficient training. In *Proceedings of the International Symposium on Computer Architecture*. 706–719.

Received 23 July 2022; revised 5 October 2022; accepted 31 October 2022